

Рабочий протокол и отчёт по лабораторной работе № 5
[C++ & UNIX]: C++ OOP / PARALLEL

```

    unordered_map<string, Float> vars;
public:
    void setVar(const string& name, Float value) { vars[name] = value; }
    Float getVar(const string& name) const {
        auto it = vars.find(name);
        if (it == vars.end())
            throw runtime_error("Непрожабил_переменную:_ " + name);
        return it->second;
    }
    bool hasVar(const string& name) const { return vars.count(name); }
};

```

Класс абстракт (вычисление своего значения в контексте), константа, переменная, бинарка

```

//Абстрактное выражение
class Expr {
public:
    virtual ~Expr() = default;
    virtual Float eval(ExecutionContext& ctx) const = 0;
};

class NumberExpr : public Expr {
    Float value;
public:
    NumberExpr(Float val) : value(val) {}
    Float eval(ExecutionContext&) const override { return value; }
};

class VariableExpr : public Expr {
    string name;
public:
    VariableExpr(const string& n) : name(n) {}
    Float eval(ExecutionContext& ctx) const override { return ctx.getVar(name); }
};

class BinaryExpr : public Expr {
    char op;
    unique_ptr<Expr> left;
    unique_ptr<Expr> right;
public:
    BinaryExpr(char oper, unique_ptr<Expr> l, unique_ptr<Expr> r)
        : op(oper), left(move(l)), right(move(r)) {}
    Float eval(ExecutionContext& ctx) const override {
        Float a = left->eval(ctx);
        Float b = right->eval(ctx);
        switch(op) {
            case '+': return a + b;
            case '-': return a - b;
            case '*': return a * b;
            case '/':
                if (b == 0) throw runtime_error("Ква!_На_нольто_не_дели");
                return a / b;
            default: throw runtime_error("Непрожабил_операцию");
        }
    }
};

```

Инструкция и иерархия, присваивание, вывод в консоль, вывод в файл, цикл

```
//Абстрактная инструкция
class Statement {
public:
    virtual ~Statement() = default;
    virtual void execute(ExecutionContext& ctx) = 0;
};

class AssignStmt : public Statement {
    string varName;
    unique_ptr<Expr> expr;
public:
    AssignStmt(const string& name, unique_ptr<Expr> e) : varName(name), expr(
        move(e)) {}
    void execute(ExecutionContext& ctx) override {
        Float val = expr->eval(ctx);
        ctx.setVar(varName, val);
    }
};

class PrintStmt : public Statement {
    unique_ptr<Expr> expr;
public:
    PrintStmt(unique_ptr<Expr> e) : expr(move(e)) {}
    void execute(ExecutionContext& ctx) override {
        Float val = expr->eval(ctx);
        safe_print("тут_эмоодзи_жабы_" + to_string(val));
    }
};

class WriteFileStmt : public Statement {
    unique_ptr<Expr> expr;
    string filename;
public:
    WriteFileStmt(unique_ptr<Expr> e, const string& fname) : expr(move(e)),
        filename(fname) {}
    void execute(ExecutionContext& ctx) override {
        Float val = expr->eval(ctx);
        ofstream file(filename, ios::app);
        if (!file) throw runtime_error("Файлик:_ " + filename + "_не_нашел");
        file << val << endl;
    }
};

class ForLoopStmt : public Statement {
    int iterations;
    vector<unique_ptr<Statement>> body;
public:
    ForLoopStmt(int iter, vector<unique_ptr<Statement>> b) : iterations(iter),
        body(move(b)) {}
    void execute(ExecutionContext& ctx) override {
        for (int i = 0; i < iterations; ++i) {
            for (auto& stmt : body) {
                stmt->execute(ctx);
            }
        }
    }
};
```

Парсер

```
//Парсер
class Parser {
    vector<string> tokens;
    size_t pos;

    string current() { return (pos < tokens.size()) ? tokens[pos] : ""; }
    void next() { ++pos; }

    //Разбор выражения
    unique_ptr<Expr> parseExpr() {
        auto left = parseTerm();
        while (true) {
            string op = current();
            if (op == "прижабь" || op == "отжабь" || op == "пережабь" || op == "
разжабь") {
                next();
                auto right = parseTerm();
                char c = (op == "прижабь") ? '+' : (op == "отжабь") ? '-' : (op
== "пережабь") ? '*' : '/';
                left = make_unique<BinaryExpr>(c, move(left), move(right));
            } else break;
        }
        return left;
    }

    //число, переменная
    unique_ptr<Expr> parseTerm() {
        string tok = current();
        if (tok.empty()) throw runtime_error("Выражение не закончено");

        //Обработка отрицательного числа : если токен начинается с минуса и дальше цифра
        if ((tok[0] == '-' && tok.size() > 1 && isdigit(tok[1])) || isdigit(tok
[0])) {
            next();
            size_t processed;
            double val = stod(tok, &processed);
            if (processed == tok.size())
                return make_unique<NumberExpr>(val);
            else
                throw runtime_error("Некорректное число: " + tok);
        }

        //Переменная
        if (isalpha(tok[0]) || tok[0] == '_') {
            next();
            return make_unique<VariableExpr>(tok);
        }

        //Выражение в скобках
        if (tok == "(") {
            next();
            auto expr = parseExpr();
            if (current() != ")") throw runtime_error("Я ждал ')'");
            next();
            return expr;
        }
        throw runtime_error("Неожиданный токен в выражении: " + tok);
    }
}

public:
```

```

vector<unique_ptr<Statement>> parseLine(const string& line) {
    //Токенизация
    tokens.clear();
    pos = 0;
    string buf;
    for (char ch : line) {
        if (isspace(ch)) {
            if (!buf.empty()) tokens.push_back(buf);
            buf.clear();
        } else if (ch == ';' || ch == '(' || ch == ')' || ch == '{' || ch ==
            '}' || ch == ',') {
            if (!buf.empty()) tokens.push_back(buf);
            tokens.push_back(string(1, ch));
            buf.clear();
        } else {
            buf += ch;
        }
    }
    if (!buf.empty()) tokens.push_back(buf);

    return parseStatements();
}

private:
//Останавливаемся на ';' или '}'
vector<unique_ptr<Statement>> parseStatements() {
    vector<unique_ptr<Statement>> stmts;
    while (pos < tokens.size() && current() != ";" && current() != "}") {
        stmts.push_back(parseOneStmt());
        if (pos < tokens.size() && current() == ";") {
            next(); //пропускаем разделитель
        }
    }
    return stmts;
}

unique_ptr<Statement> parseOneStmt() {
    //Присваивание: проверяем, что следующий токен это "="
    if (pos + 1 < tokens.size() && tokens[pos+1] == "=") {
        string var = current();
        next(); //имя
        next(); //'='
        auto expr = parseExpr();
        return make_unique<AssignStmt>(var, move(expr));
    }
    //Проквакай
    if (current() == "проквакай") {
        next();
        auto expr = parseExpr();
        return make_unique<PrintStmt>(move(expr));
    }
    //Закувшинь
    if (current() == "закувшинь") {
        next();
        auto expr = parseExpr();
        if (current() != "в") throw runtime_error("ждал_предлог_в',_неуч");
        next();
        if (pos >= tokens.size()) throw runtime_error("А_имя_файла_кто_укажет?");
        string fname = current();
        next();
    }
}

```

```

        return make_unique<WriteFileStmt>(move(expr), fname);
    }
    //Повтори
    if (current() == "повтори") {
        next();
        if (current() != "(") throw runtime_error("Ждал_( ' ' после повтори");
        next();
        if (pos >= tokens.size()) throw runtime_error("А_квакаты_мне_сколь_ко?");
        int iter = stoi(current());
        next();
        if (current() != ")") throw runtime_error("Ждал_') '");
        next();
        if (current() != "{") throw runtime_error("Ждал_тело_цикла_в_{_}");
        next(); //пропускаем '{'
        auto body = parseStatements(); //теперь корректно остановится на '}'
        if (current() != ")") throw runtime_error("Ждал_'} ' в_конце_цикла");
        next(); //пропускаем '}'
        return make_unique<ForLoopStmt>(iter, move(body));
    }
    throw runtime_error("Не_понял_команду:_ " + current());
}
};

```

Многопоточность

```

//Поток выполнения
class FrogThread {
    int id;
    vector<unique_ptr<Statement>> program;
    chrono::steady_clock::time_point start_time, end_time;
    bool finished;
public:
    FrogThread(int idx, vector<unique_ptr<Statement>> prog) : id(idx), program(
        move(prog)), finished(false) {}

    void run() {
        auto sys_start = chrono::system_clock::now();
        time_t start_t = chrono::system_clock::to_time_t(sys_start);
        string start_str = ctime(&start_t);
        start_str.pop_back();
        safe_print("тут_модзи_жабы_Поток[_ " + to_string(id) + "]_СТАРТ_в_" +
            start_str);

        start_time = chrono::steady_clock::now();

        try {
            ExecutionContext ctx;
            for (auto& stmt : program) {
                stmt->execute(ctx);
            }
        } catch (const exception& e) {
            safe_print("тут_модзи_жабы_Поток[_ " + to_string(id) + "]_ОШИБКА:_ "
                + e.what());
        }

        end_time = chrono::steady_clock::now();
        auto sys_end = chrono::system_clock::now();
        time_t end_t = chrono::system_clock::to_time_t(sys_end);
    }
};

```

```

        string end_str = ctime(&end_t);
        end_str.pop_back();
        safe_print("тут_эמודзи_жабы_Поток[" + to_string(id) + "]_ФИНИШ_В_" +
            end_str);
        finished = true;
    }

    bool isFinished() const { return finished; }
};

```

Мейн

```

int main() {
    safe_print("тут_эמודзи_жабы_FrogLang_v2.1_тут_эמודзи_жабы");
    safe_print("Для_завершения_ввода_введите_пустую_строку");

    vector<string> rawLines;
    string input;
    int lineNum = 1;
    while (true) {
        cout << "тут_эמודзи_жабы_строка_" << lineNum << "_>_";
        getline(cin, input);
        size_t first = input.find_first_not_of("\t");
        if (first == string::npos) break;
        input = input.substr(first);
        rawLines.push_back(input);
        ++lineNum;
    }

    if (rawLines.empty()) {
        safe_print("Нет_команд._Лягушка_ускакала...");
        return 0;
    }

    Parser parser;
    vector<vector<unique_ptr<Statement>>> allPrograms;
    bool parseError = false;

    for (size_t i = 0; i < rawLines.size(); ++i) {
        try {
            auto prog = parser.parseLine(rawLines[i]);
            allPrograms.push_back(move(prog));
        } catch (const exception& e) {
            safe_print("Ошибка_парсинга_строки_" + to_string(i+1) + ":__" + e.what()
                ());
            parseError = true;
        }
    }

    if (parseError) {
        safe_print("Иза-ошибок_парсинга_выполнение_невозможно.");
        return 1;
    }

    // Запуск потоков
    vector<unique_ptr<FrogThread>> threads;
    vector<thread> workers;
    for (size_t i = 0; i < allPrograms.size(); ++i) {
        auto ft = make_unique<FrogThread>(i+1, move(allPrograms[i]));
        workers.emplace_back([&ft = *ft]() { ft.run(); });
    }
}

```

```

        threads.push_back(move(ft));
    }

    for (auto& w : workers) {
        if (w.joinable()) w.join();
    }

    safe_print("Все потоки завершены тут э модзи жабы");
    return 0;
}

```

Результат:

```

FrogLang v2.1 Для завершения ввода введите пустую строку строка
1 > x = 4; повтори(4) {x = x прижабь 1; проквакай x; } строка
2 > x = 5; закувшинь x в log.txt строка
3 > Поток
[ 1] СТАРТв Sat May 16 11:10:09 2026 Поток
[ 2] СТАРТв Sat May 16 11:10:09 2026
5.000000
6.000000
7.000000
8.000000 Поток
[ 1] ФИНИШв Sat May 16 11:10:09 2026 Поток
[ 2] ФИНИШв Sat May 16 11:10:09 2026 Все потоки завершены

```

Заключение

В ходе работы были изучены:

- написание своего языка
- основы ООП